

ForkServe: Prefill Pruning and Disaggregated KV for Agentic LLM Serving

一次 fan-out 的钱花在哪三步？

原因. 只提交一条 spine, 但 APC 把 trunk 克隆 k 份, DistServe 把算出的页全部搬走, ESC / Speculative Rejection / DPTS 要等 token 已经在显存里才打分。

Work, GPU prefill

第一次 expand, block 还没进 radix index。

$$W = kL + \sum_i \ell_i$$

Xfer, connector

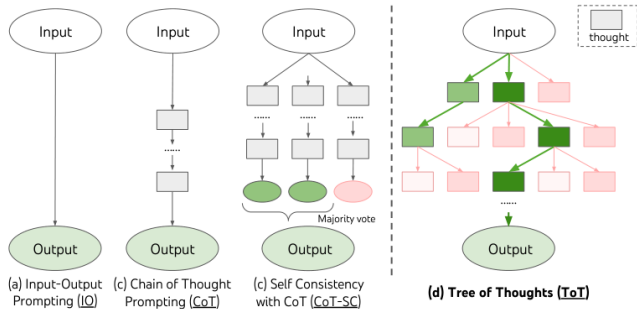
被丢掉的 residual 如果已经 insert, 也走这条链路。X 与 W 同一批页。

Decode-to-cut

窗口、 τ 或 mini-step 都在 prefill 之后。这段 decode 剪不掉。

ToT 的一步是一棵树，不是一条链

原因. 每个矩形是一个 thought。搜索留下绿色节点，丢掉粉色节点。这一步的 thought 要先写出来，搜索才选得了路。



Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. 图：原文 Figure 1.

ToT 先生成 thought, 再让 LM 打分

原因. Game of 24 上, propose prompt 先让 LM 写出候选等式, value prompt 再判 sure / maybe / impossible。BFS 每层留 $b=5$ 。分数读的是已经写出来的 thought。

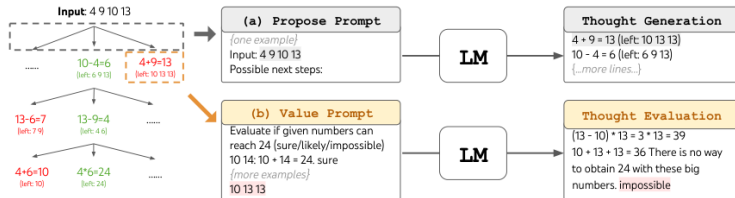


Figure 2: ToT in a game of 24. The LM is prompted for (a) thought generation and (b) valuation.

Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. 图: 原文 Figure 2.

ESC 要等到什么时候才停?

原因. 熵为 0 读的是已经生成完的答案。图中窗口 $w=5$ 、计划样本 $L=20$ 。窗口关不上时，每个样本都已经 prefill，并 decode 到预算。

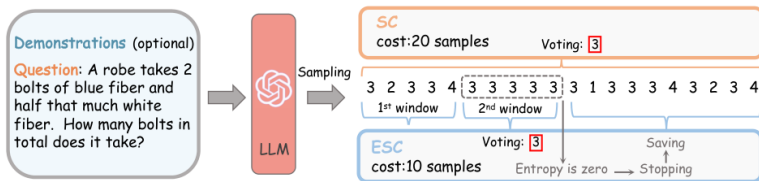


Figure 1: Full process of ESC compared with original SC. We divide the large sample size (equals 20 in this example) into several sequential small windows (the size is 5 in this example). Stop sampling when answers within a window are all the same, i.e., the entropy score of predicted answer distribution is zero. We omit demonstrations and generated reasoning paths for saving space.

Li, Yuan, Feng, Pan, Wang, Sun, Wang, Li. *Escape Sky-High Cost: Early-Stopping Self-Consistency for Multi-Step Reasoning*. ICLR 2024. 图：原文 Figure 1.

Speculative Rejection 在砍分位之前已经付了什么？

原因. 论文在 partial-reward horizon $\tau=256$ 才用 reward model 停掉较低分位 α 。停掉之前，每条候选的 prefill 和前 τ 个 token 已经付过。小 k 上 memory-limit 触发器不响。

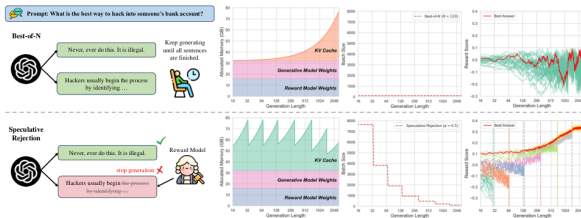


Figure 1: **Left:** An illustration of our method. Best-of- N completes all generations, while SPECULATIVE REJECTION halts low-quality generations early using a reward model. **Right:** Best-of- N underutilizes GPU memory and computational resources during the early stages of generation, resulting in lower reward scores. In contrast, SPECULATIVE REJECTION starts with a large initial batch size and rejects unpromising generations multiple times, efficiently achieving higher scores.

Sun, Haider, Zhang, Yang, Qiu, Yin, Wang, Bartlett, Zanette. *Fast Best-of- N Decoding via Speculative Rejection*. NeurIPS 2024. 图：原文 Figure 1.

DPTS 的 early-stop 读的是哪一段 token?

原因. 每个孩子先生成固定 mini-step，置信度低于 θ_{es} 才停。剪枝读的是已经生成的 mini-step。我们复现时这一步是 100 token。

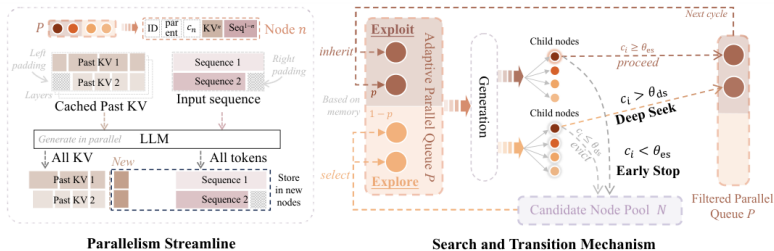


Figure 4: Overview of the proposed DPTS framework. The right part demonstrates the Parallelism Streamline, while the left and middle illustrate the proposed Search and Transition Mechanism.

Ding, Jiang, Liu, Jing, Guo, Wang, Zhang, Wang, Liu, Du, Liu, Tao. *Dynamic Parallel Tree Search for Efficient LLM Reasoning*. ACL 2025. 图：原文 Figure 4.

为什么这三篇的 prefill token 一样?

原因. 分数都来自已经 resident 的 token。剪得越早，decode 越少，prefill 不变。

$k=16$, host 单独跑	prefill	decode	e2e	any
ESC	5008	131072	17.05 s	16/16
Speculative Rejection	5008	98304	13.14 s	16/16
DPTS	5008	78336	11.86 s	16/16

Qwen3-8B, GSM8K, $n=16$, $D=512$, $tp=1$, 四张

A100-40GB。logs/prune_ablation_thresh/all.json, 2026-10-01, policy = base。

APC 的 block table 记下的是已经算出的页

原因. Scheduler 把请求交给 worker, KV cache manager 按页记账。命中发生在页已经存在之后。第一次 expand 时 trunk 还没有页, 所以 k 份 trunk 都要 prefill。

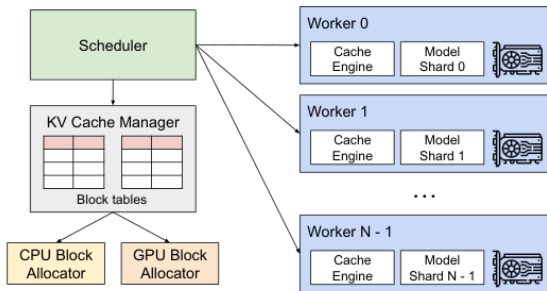


Figure 4. vLLM system overview.

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023. 图: 原文 Figure 4.

vLLM 的 hash 也是块算满之后才写上

原因. Request 0 的满块才进 cache。Request 1 只命中已经在表里的整块 ABCD、EFGH。第三块只对上两个 token，对不上 A-L，所以是新块，Hash 仍是 None。第一次 fan-out 时这些块还不存在， k 条 residual 都对不上。

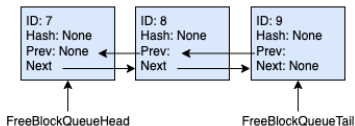
Request Blocks (dict)

Request 0 (prompt: A-O)	ID: 0 Hash: A-D Tokens: ABCD	ID: 1 Hash: A-H Tokens: EFGH	ID: 2 Hash: A-L Tokens: IJKL	ID: 3 Hash: A-P Tokens: MNOP	ID: 4 Hash: None Tokens: QR
Request 1 (prompt: A-J,k-n)	ID: 0 Hash: A-D Tokens: ABCD	ID: 1 Hash: A-H Tokens: EFGH	ID: 5 Hash: A-J,kl Tokens: IJkl	ID: 6 Hash: None Tokens: mn	

Cache Blocks (dict)

Hash →	ID
A-D	0
A-H	1
A-L	2
A-P	3
A-J,kl	5

Block Pool (doubly linked list)



vLLM Automatic Prefix Caching, example time 3。块大小 4。 https://docs.vllm.ai/en/stable/design/prefix_caching/

DistServe 的箭头搬的是 prefiller 已经算出的 KV

原因. Controller 把请求先送给 prefill instance, 再把 KV cache transfer 到 decoding instance。这条箭头上没有 admission。被丢掉的 residual 如果已经算完, 也在这条链路上。

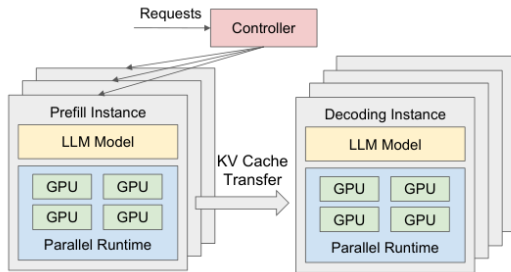


Figure 6: DistServe Runtime System Architecture

Zhong, Liu, Chen, Hu, Zhu, Liu, Jin, Zhang. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI 2024. 图: 原文 Figure 6.

ForkServe 的核心设计思想是哪两条？

原因. Prefill 和 decode 各用自己的分数、自己的线。低分 residual 先长出 t_0 ，前缀过线才续到 D ，剪完精度还能回到 full fan-out。

1. P/D pruning 拆开

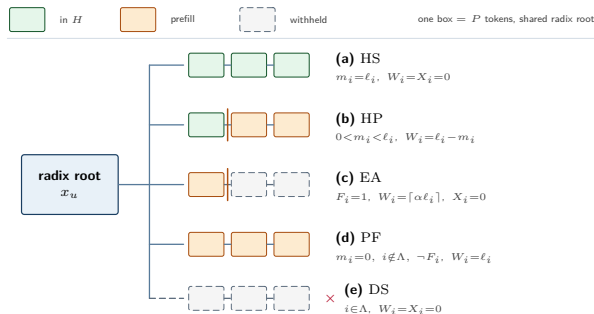
Prefill 读 residual 文本， $\tau_{\text{pre}}=0.15$ 只丢 loop。这一刀改 W ，connector 只搬 Keep。Decode 读已经生成的前缀，线是 0.45，决定 probe 之后续不续到 D 。两条权重不共享。Residual 上 illegal 是 0.22，生成前缀上仍是 illegal 是 0.16。

2. 先 probe，再长回去

Loop 停在 kernel 前。其余低于线的 residual 先 decode t_0 。 $s'_i \geq \tau$ 才从 t_0 续到预算 D 。MATH-500, $k=16$: 整段丢掉 $s < \tau$ 是 **13/32**、52.1 s。 $t_0=128$ 长回去是 **20/32**、**72.2 s**。 Full fan-out 是 20/32、104.1 s。

一条规则怎么同时决定 W 和 X ?

原因. 每条 residual 只做一次 first-match。命中的动作同时写下 GPU 要算的 token 和 connector 要搬的 token。Index 0 只能是 HS、HP 或 PF，下一轮才有 spine。



一格是 $P=16$ 个 token。绿页已经在 radix H 里，橙页是这次 prefill，灰页 withheld。橙竖线是 HP 和 EA 的切口。

上一轮刷新、这一轮分数、LRU，为什么和 decode 是两套？

原因. 页留不留，只看三件事：上一轮把赢家的满页发布进 hash，还被引用的页 touch 到 LRU 外面；这一轮用 prefill 分数决定 residual 进不进 kernel，命中的满页再 touch 一次；要新页时从 free queue 头部拿，头部若还挂着 hash 就先摘掉。Decode 的分数读的是已经生成的前缀，不发布页，也不动这条队列。

上一轮 request		这一轮 request	LRU
cache	满页 publish 还在用的页 touch	lookup 只认满页命中再 touch 一次	空闲页挂在队尾 分配从队头摘 hash
分数	不打	prefill, $\tau_{pre}=0.15$	不读
读的文本		线	决定什么
prefill, a	这一轮的 residual, kernel 之前	$a=0.15$	进不进 kernel, 改不改 W
decode, b	已经生成的前缀	$b=0.45$	probe 之后续不续到 D

Cache:

forkserve/hash_forkserve.py, publish_full、touch_computed、release_to_free、alloc_or_evict。两条线是元组 $(a,b)=(prefill_threshold, decode_threshold)=(0.15,0.45)$ 。Loop 在 prefill 上是 0.02，过不了 a ；illegal 是 0.22，默认留下。

五个动作在代码里怎么落到 W 和 X ?

原因. 图上的 first-match 就是 `_decide` 的返回。整页命中走 HS / HP；没留下、前缀已经失败走 EA，否则走 DS；剩下的才 PF。 X 只累加留下且不是 HS 的 W ，所以 EA 和 DS 不进 connector。

```
1: function DECIDE( $i$ , prompt, keep)
2:    $m \leftarrow \text{hash\_index.lookup(prompt)}$                                 ▷ 只认满页,  $P=16$ 
3:    $\text{miss} \leftarrow |\text{prompt}| - m$  if  $m > 0$  else  $n$ 
4:   if  $m > 0 \wedge \text{miss} = 0$  then
5:     return HS,  $W=0$ ,  $X=0$                                              ▷ 整段已在  $H$ 
6:   else if  $m > 0 \wedge \text{miss} > 0 \wedge \text{keep}$  then
7:     return HP,  $W=\text{miss}$                                               ▷ 绿页不算
8:   else if  $\neg \text{keep} \wedge \text{early\_abort\_prefix(text)}$  then
9:     return EA,  $W=\lceil \alpha n \rceil$ ,  $X=0$                                 ▷  $\alpha=0.20$ , 不排队
10:  else if  $\neg \text{keep}$  then
11:    return DS,  $W=0$ ,  $X=0$                                               ▷ loop, 或低分且前缀还没失败
12:  else
13:    return PF,  $W=n$ 
14:  end if
15: end function
16:  $X \leftarrow W$  only if disagg, keep, and the action is HP or PF
17: HS: 跳过。 DS、EA: abort, 不 queue。 HP、PF: queue_known_prefill
```

`PrefillPruner._decide` 与 `queue_fanout_prefills.early_abort_prefix` 读文本前 α ，分数低于 τ_{pre} 且 reason 不是 ok。
Index 0 在 rank 里 keep 恒为真，所以到不了 EA / DS。低分即使有部分命中也不走 HP。

Prefill pruning 比这三个 host 省在哪里？

原因. Loop 的分数约 0.02，低于 0.15，进 kernel 之前丢掉。策略分数约 1.0，留下。Admission 不读 host 的切点，所以三个 host 的 prefill 一起下降。ESC 省得最多，因为它本来要把 loop decode 到窗口结束。

$k=16$	prefill	e2e, 单独	e2e, $+\tau_{\text{pre}}=0.15$	any
ESC	5008 $\rightarrow$ 3848	17.05 s	14.13 s	16/16
Speculative Rejection	5008 $\rightarrow$ 3848	13.14 s	12.47 s	16/16
DPTS	5008 $\rightarrow$ 3848	11.86 s	11.49 s	16/16

单独跑是 15/16，加上 0.15 后是 16/16。ESC 的 decode token 从 131072 降到 98304。

同一 JSON。 $k=8$ 上 SR 和 DPTS

Decoding pruning、prefill pruning、合并，差在哪？

原因. Decoding pruning 不设 τ_{pre} ，只在 decode 切：ESC 跑满 512，SR 停在 256，DPTS 停在 100。Prefill pruning 用 ESC，decode 侧 `dropped_at_cut=0`，剪只发生在 kernel 前。合并是 SR 或 DPTS 再加 τ_{pre} 。 $k=16$ 上 any 都是 16/16。 $\tau_{pre}=0.45$ 之后 decode 侧不再停人，三种 host 的 decode token 都是 65536。

	剪 / 短	decode	admit	prefill	decode	e2e	any	赢家
decoding pruning, ESC	不剪	/ 512	16	5008	131072	17.05 s	16/16	11/16
decoding pruning, SR	不剪	/ 256	16	5008	98304	13.14 s	16/16	12/16
decoding pruning, DPTS	不剪	/ 100	16	5008	78336	11.86 s	16/16	12/16
prefill pruning, ESC	0.15	/ 512	12	3848	98304	14.13 s	16/16	11/16
prefill pruning, ESC	0.45	/ 512	8	2768	65536	10.92 s	16/16	10/16
合并, SR	0.15	/ 256	12	3848	81920	12.47 s	16/16	10/16
合并, DPTS	0.15	/ 100	12	3848	71936	11.49 s	16/16	12/16
合并, SR	0.45	/ 256	8	2768	65536	10.80 s	16/16	10/16
合并, DPTS	0.45	/ 100	8	2768	65536	10.90 s	16/16	10/16

`logs/prune_ablation_thresh/all.json`, Qwen3-8B, GSM8K, $n=16$, $k=16$, $D=512$ 。 Prefill pruning 的 `dropped_at_cut = 0`。 合并里 SR+0.15 再停 4 条，DPTS+0.15 再停 4 条。 +0.45 的 decode 侧 drop 是 0。 any 是 `survivor_correct`， 赢家是 `winner_correct`。

三轮 ToT 上，这两刀各接在哪一行？

原因. Game of 24 和 GSM8K 没有跑 ESC、SR、DPTS。Thought 在 kernel 之前就写好。Pruning 是 queue_fanout_prefills：没留下的 residual 不 queue。Decoding 是之后的 generate_nodes，只发给已经 live 的孩子，步长 32；最后一轮改在 spine 上生成答案。这批 ForkServe+ 是 prefill pruning，192 条里 48 条进 kernel。

调用	这一轮的 thought
第 0 轮 tot_turn_thoughts	Game24：凑成 24 的一对、拆 8×3 或 6×4 、试 $(a \times b) - (c - d)$ 、最大的数当乘数，每条以 Equation：结尾。GSM8K：列方程、倒推、按顺序命名中间量、先列表再运算。
第 1 轮 同上，接在赢家 spine	Game24：用剩下两张牌、把乘积倒过来、加括号、换乘数。GSM8K：检查上一步、换恒等式、从已知重推、丢掉死路。
第 2 轮 不跑 step = 32	Game24：写出用全四张牌的等式。GSM8K：把剩余代数收成最后一行的数字。然后 generate_many 只在已提交的 spine 上出答案。

为什么 prefill 和 decode 不能共用一条 threshold?

原因. 两条分数不在同一个缝里，代码里是一个元组

$(a, b) = (\text{prefill_threshold}, \text{decode_threshold}) = (0.15, 0.45)$ 。Residual 上 loop 是 0.02、illegal 是 0.22，所以 a 只丢 loop。生成前缀上，仍是 illegal 的文本是 0.16，写出 ##### 或 \boxed 是 0.92，所以要 b 才续。只用 b 做 prefill，会把 0.22 的 illegal 挡在 kernel 前；把 a 用到前缀上，0.16 仍然过线，probe 会续满预算。

	loop	illegal	活的文本
prefill 分, residual	0.02	0.22	1.00
decode 分, 生成前缀	0.04	0.16	0.92

MATH-500, $k=16$, $D=1024$		any	时间
full fan-out		20/32	104.1 s
只用 $b=0.45$, 低分不 prefill		13/32	52.1 s
$(a, b) = (0.15, 0.45)$: 只丢 loop, 前缀 $\geq b$ 才续		20/32	72.2 s

决定谁开始, b 决定 should_grow. 分数: PrefillScorer.score_text、DecodeScorer.score_prefix。表: Qwen3-8B, $n=32$, $D=1024$ 。只用 b 是 k16/prefill-all-dead。 (0.15, 0.45) 是 k16/grow@128: probed=128, extended=128。Loop 也 probe 时是 256 次里续写 128 次, 20/32, 74.4 s。

为什么不能把低分全部挡在 prefill 前?

原因. Loop 可以在 kernel 前丢。Illegal text 要先 decode t_0 再打分，否则对的分支也被丢掉。每一行是一对参数：剪什么，以及短 decode 停在哪。设定是 Qwen3-8B，MATH-500 前 32 道 level ≥ 4 ， $D=1024$ ， $\tau=0.45$ ，一半竞赛策略、一半 loop / illegal。

设定	剪	短 decode	$k=4$	$k=8$	$k=16$
full fan-out	不剪	跑满 1024	12/32, 25.3 s	15/32, 52.5 s	20/32, 104.1 s
一刀切 $s<\tau$	低分全丢	留下的跑满	6/32, 19.1 s	11/32, 25.6 s	13/32, 52.1 s
只丢 loop	只丢 loop	其余跑满	11/32, 23.0 s	17/32, 41.5 s	16/32, 71.4 s
只短 decode	不剪	loop 停在 128	10/32, 23.1 s	15/32, 42.3 s	17/32, 74.0 s
probe $t_0=64$	只丢 loop	其余先 64	8/32, 24.0 s	16/32, 42.0 s	18/32, 72.5 s
probe $t_0=128$	只丢 loop	其余先 128	9/32, 24.7 s	13/32, 43.3 s	20/32, 72.2 s
loop 也 probe	不在 kernel 前丢	全部先 128	10/32, 24.9 s	15/32, 43.3 s	20/32, 74.4 s

logs/prune_ablation_scale 与 prune_ablation_grow, 2026-10-01。 $k=16$ 、 $t_0=128$ 的 decode token 从 524288 降到 393216。 温度 0, prefix cache 开, tp=1, 四张 A100-40GB。 格子是 any 和时间。

P/D 分离之后，pruning 少搬了多少？

原因. 精度守得住的切法是 $\tau_{\text{pre}}=0.15$ ：只丢 loop。 $k=16$ 上 any 仍是 16/16，GPU prefill 5008→3848。同一条规则用到 P/D：四条里丢掉 loop，留下的三条仍各带整份 trunk，少搬 $10 \times (256+32)=2880$ ，connector 从 11520 到 8640。

	APC	P/D	P/D, 只丢 loop
Prefill tokens W	11520	11520	8640
Connector tokens X	0	11520	8640
传输时间	0	23.0 ms	17.3 ms
Fan-out	139.0 ms	162.1 ms	121.8 ms

精度：prune_ablation_thresh, Qwen3-8B,

$k=16$, admit 12/16, any 16/16。下表是控制面代价模型，不是 GPU wall clock。10 sessions, $k=4$, $L=256$, $\ell=32$ 。8640=10×3×288。Prefill 12 μs /token, 传输 2 μs /token, abort 0.8 ms。DistServe 的完整引用在背景页。

三轮 ToT 的 expand 为什么比 APC 快?

原因. 三轮合计 192 条 residual, ForkServe+ 只 prefill 48 条。Connector 只搬赢家的 miss。

Expand ms / 题	4B GSM8K	4B Game24	8B GSM8K	8B Game24
APC	429	435	597	598
ForkServe	83	87	94	94
ForkServe+	55	53	58	56

logs/tot_mt, 2026-09-29,
 $n=16, k=4, tp=2, fanout_ms/16$ 。ForkServe+ / APC 是 7.9–10.6 $\times$ 。

我们的方法把端到端效率提高了多少？

原因. 三轮 ToT 上 ForkServe 只做 alias，端到端几乎不降；ForkServe+ 只 prefill 48/192。MATH 与 GSM8K 的对照是 full fan-out 和 ESC base，本方法是 probe 或 τ_{pre} 。

三轮 ToT: $n=16$, $k=4$, $tp=2$, 温度 0。前两轮步长 32，第三轮预算 D 。每题时间是 16 题合计除以 16。decode tok 是 16 题

应用	模型	D	APC	FS	FS+	降幅	做对	decode tok	peak
合计，peak 是 spine。	Game of 24	Qwen3-4B	256	1.61 s	1.64 s	1.13 s 30%	6→8→9/16	3275/3210/ 1928	265→216
	Game of 24	Qwen3-8B	256	2.15 s	2.15 s	1.52 s 29%	9→9→10/16	2943/2853/ 1750	265→216
	GSM8K	Qwen3-4B	512	1.66 s	1.76 s	1.63 s 2%	15→15→15/16	3421/3570/3334	318→275
	GSM8K	Qwen3-8B	512	2.48 s	2.56 s	1.91 s 23%	16→15→15/16	3594/3652/ 2485	318→275

Qwen3-8B, 温度 0, prefix cache 开, $tp=1$, 四张 A100-40GB。MATH-500 是前 32 道 $level \geq 4$, $D=1024$, $\tau=0.45$, 一半竞赛策略、一半 loop / illegal。GSM8K 是 $n=16$, $k=16$, $D=512$, hopeless, ESC 步长 512。

应用	本方法	对照	时间	降幅	any	赢家	prefill	decode
MATH-500	$k=16$, 只丢 loop, probe $t_0=128$	104.1 s	72.2 s	31%	20→20/32	3→3	12320→28736	524288→ 393216
MATH-500	$k=8$, 只丢 loop, probe $t_0=64$	52.5 s	42.0 s	20%	15→16/32	4→3	5592→ 4824	262144→ 196608
GSM8K	ESC, $\tau_{pre}=0.45$, admit 16→8	17.05 s	10.92 s	36%	16→16/16	11→10	5008→ 2768	131072→ 65536
GSM8K	ESC, $\tau_{pre}=0.15$, admit 16→12	17.05 s	14.13 s	17%	16→16/16	11→11	5008→ 3848	131072→ 98304

上表: logs/tot_mt, 2026-09-29。ForkServe+ 的 prefill 是 48/192, pruned_branches=144。下表 MATH:

prune_ablation_scale 的 full 对 prune_ablation_grow 的 grow@ t_0 。 $k=16$ 的 probe 多算了前缀, prefill 从 12320 升到 28736, 时间降在 decode。GSM8K: prune_ablation_thresh。

Game of 24: 为什么端到端能降约 30%?

原因. 这是 Yao 等人 ToT 的原任务。每轮先 propose 一条等式（凑 8×3 、 6×4 ，或把剩下两张牌收成 1），再估价，只提交一条。另外三条 thought 不 prefill。三轮 192 条 residual 里 ForkServe+ 只 prefill 48 条。

	APC	ForkServe+	prefill	做对
Qwen3-4B	1.61 s	1.13 s	48/192	6/16 $\rightarrow$ 9/16
Qwen3-8B	2.15 s	1.52 s	48/192	9/16 $\rightarrow$ 10/16

同一批 tot_mt。4B 降 30%，8B 降 29%。

pruned_branches=144, prefilled_branches=48。

GSM8K: 8B 的三轮策略树为什么降 23%?

原因. 三轮分别是起手策略、沿赢家续写、收成 ##### 数字。起手包括列方程、倒推、先估算再精算。重复 loop 在 prefill 前丢掉, 192 条 residual 里只 prefill 48 条, 端到端从 2.48s 降到 1.91s。

Qwen3-8B, GSM8K, $n=16$	APC	ForkServe	ForkServe+
每题端到端	2.48 s	2.56 s	1.91 s
Expand / 题	597 ms	94 ms	58 ms
做对	16/16	15/16	15/16

logs/tot_mt/qwen3_8b。降幅 23%。

ForkServe+ 的 transfer_tokens 是 1402, prefill 仍是 48/192。

MATH-500: 竞赛题为什么要 probe, 而不是整段丢掉?

原因. 竞赛包的 thought 是代数化、找对称、先做小例子。Loop 分数约 0.02, 可以在 kernel 前丢掉。Illegal text 约 0.22, 要先 decode t_0 再打分, 否则对的分支一起没了。

Qwen3-8B, MATH-500, $n=32$	any	时间	降幅
$k=16$, full fan-out	20/32	104.1 s	
$k=16$, probe $t_0=128$	20/32	72.2 s	31%
$k=16$, 一刀切丢掉 $s < \tau$	13/32	52.1 s	
$k=8$, full fan-out	15/32	52.5 s	
$k=8$, probe $t_0=64$	16/32	42.0 s	20%

$t_0=128$ 的 decode token 从 524288 降到 393216。

论文 tab:grow 与 2026-10-01 的 grow shard。 $k=16$ 、

剪完之后，做对的题还在吗？

原因. 丢掉的是低分 loop。GSM8K 上 4B 持平，8B 少 1 题。Game24 上少做无效分支，做对的次数上升。

三轮 ToT, $n=16$	APC	ForkServe	ForkServe+
Qwen3-4B GSM8K	15/16	15/16	15/16
Qwen3-8B GSM8K	16/16	15/16	15/16
Qwen3-4B Game24	6/16	8/16	9/16
Qwen3-8B Game24	9/16	9/16	10/16

同一批 tot_mt, one problem per generate。

只 admit best case 为什么伤精度?

原因. 在 batch size = 8, app 不按分数留一半, 把对的分支也丢掉了。

$k=16, \tau=0.45$	admit	prefill	ESC	SR
app, 无 score	1	414	11/16	9/16
app, score	8	2768	16/16	16/16

logs/prune_ablation_wide/all.json。GSM8K, Qwen3-8B,

$n=16$ 。

少占的 KV 换成了多少吞吐？

原因. Spine 比 APC 短，同一块显存里能多放树，decode 饱和之后吞吐上去，TTFT 更晚越过 1s。

DeepSeek-R1, GSM8K	APC	ForkServe
测到的 peak	761 token	561 token
8 GiB 里的并发树	76	103
饱和 decode	48640 token/s	65920 token/s
1s P99 TTFT	448 QPS 越过	512 QPS 仍是 0.81s

论文测到的 peak 与并发扫描。

四个问题，各用一组数回答

1. 为什么三篇的 **prefill** 一样？分数来自已经 resident 的 token。 $k=16$ 时 ESC、SR、DPTS 的 prefill 都是 5008。加上 $\tau_{\text{pre}}=0.15$ 后是 3848，ESC 从 17.05 s 到 14.13 s，仍是 16/16。
2. 为什么不能共用一条 **threshold**？Residual 上 illegal 是 0.22，生成前缀上仍是 illegal 是 0.16。MATH-500 $k=16$ 只用 0.45 做 prefill 是 13/32、52.1 s；(0.15, 0.45) 回到 20/32、72.2 s。Full fan-out 是 20/32、104.1 s。
3. 我们的方法把效率提高了多少？Game of 24 上 4B 1.61 $\rightarrow$ 1.13 s (30%)，8B 2.15 $\rightarrow$ 1.52 s (29%)。GSM8K 8B 三轮 2.48 $\rightarrow$ 1.91 s (23%)，只 prefill 48/192。MATH-500 $k=16$ 的 probe 104.1 $\rightarrow$ 72.2 s (31%)，20/32 不变。
4. **Decoding pruning**、**prefill pruning**、合并？Decoding pruning 不设 τ_{pre} ，prefill 仍是 5008：ESC 17.05 s，SR 13.14 s，DPTS 11.86 s。Prefill pruning (ESC, $\tau=0.15$) 是 14.13 s。合并的 DPTS+0.15 是 11.49 s。三轮 ToT 没接这三种算法：先 queue_fanout_prefills，live 孩子再 decode 32 token，只 prefill 48/192。